Lab 4,5,6 Report

Niranjan Subash

240958011

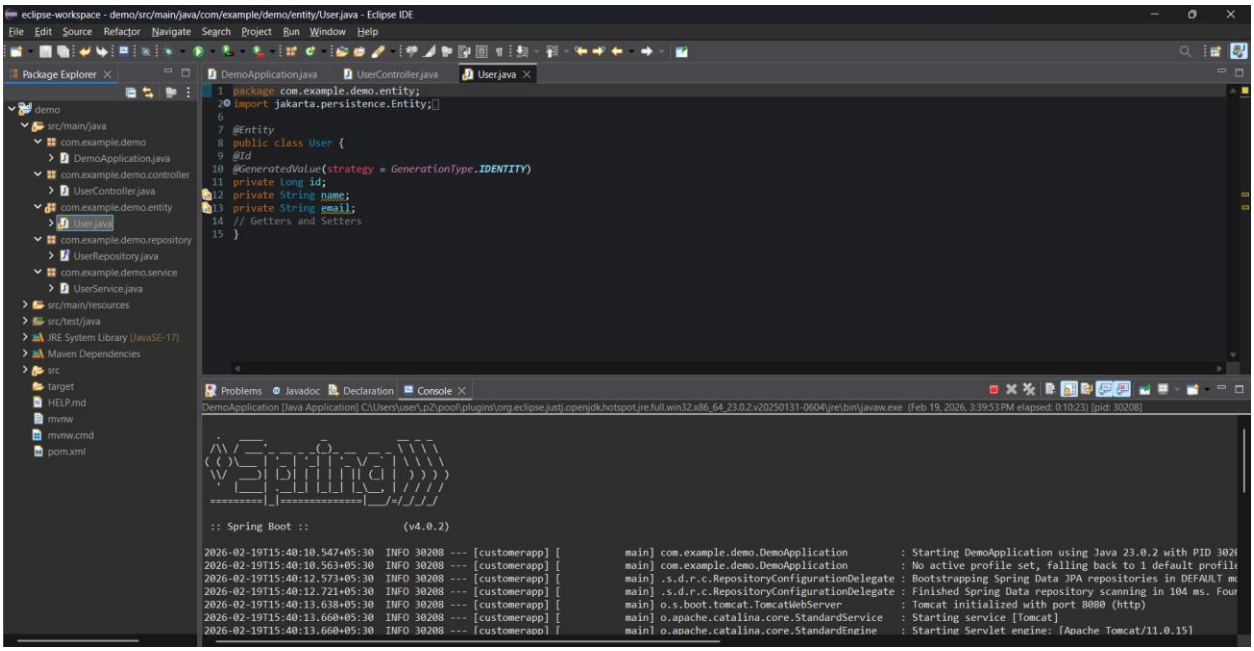
Lab 4: -

Aim

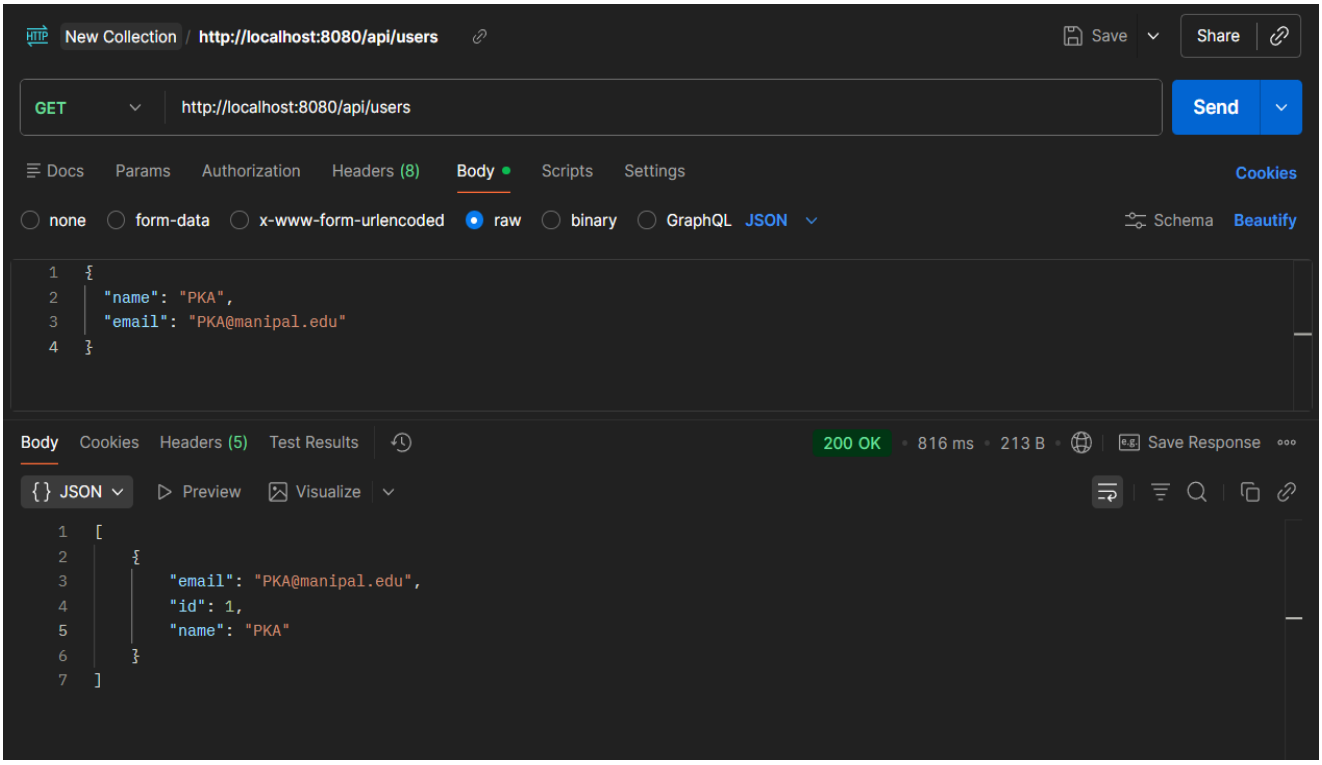
To study REST API design principles, layered architecture, and Spring Boot flow architecture used to build RESTful web services.

Objective

- To understand REST API concepts and HTTP methods.
- To learn about statelessness, status codes, versioning, and filtering.
- To understand layered architecture.
- To study the request flow in a Spring Boot application.



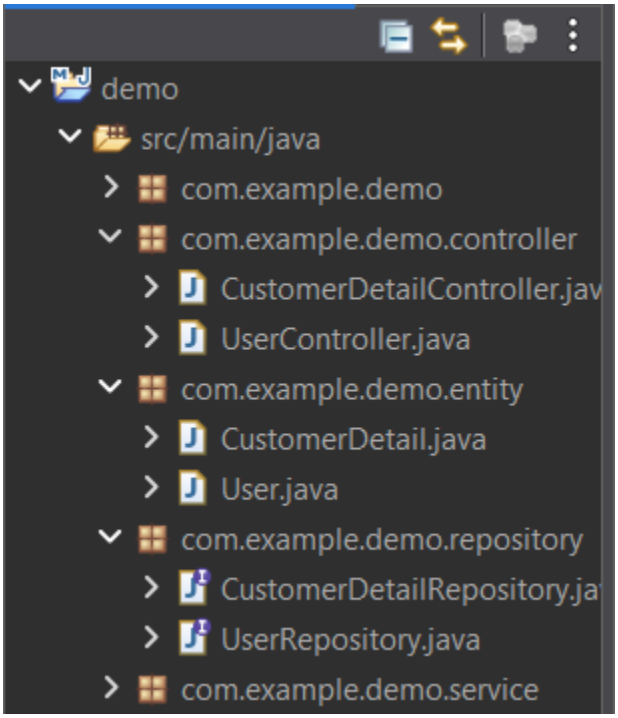
```
mysql> CREATE DATABASE springboot_db;
Query OK, 1 row affected (0.14 sec)

mysql> CREATE USER 'PKA'@'localhost' ;
Query OK, 0 rows affected (0.11 sec)

mysql> ALTER USER 'PKA'@'localhost' IDENTIFIED WITH mysql_native_password BY
    -> 'Password' ;
Query OK, 0 rows affected (0.03 sec)

mysql> GRANT ALL PRIVILEGES ON springboot_db.* TO 'PKA'@'localhost';
Query OK, 0 rows affected (0.01 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.03 sec)

mysql>
```

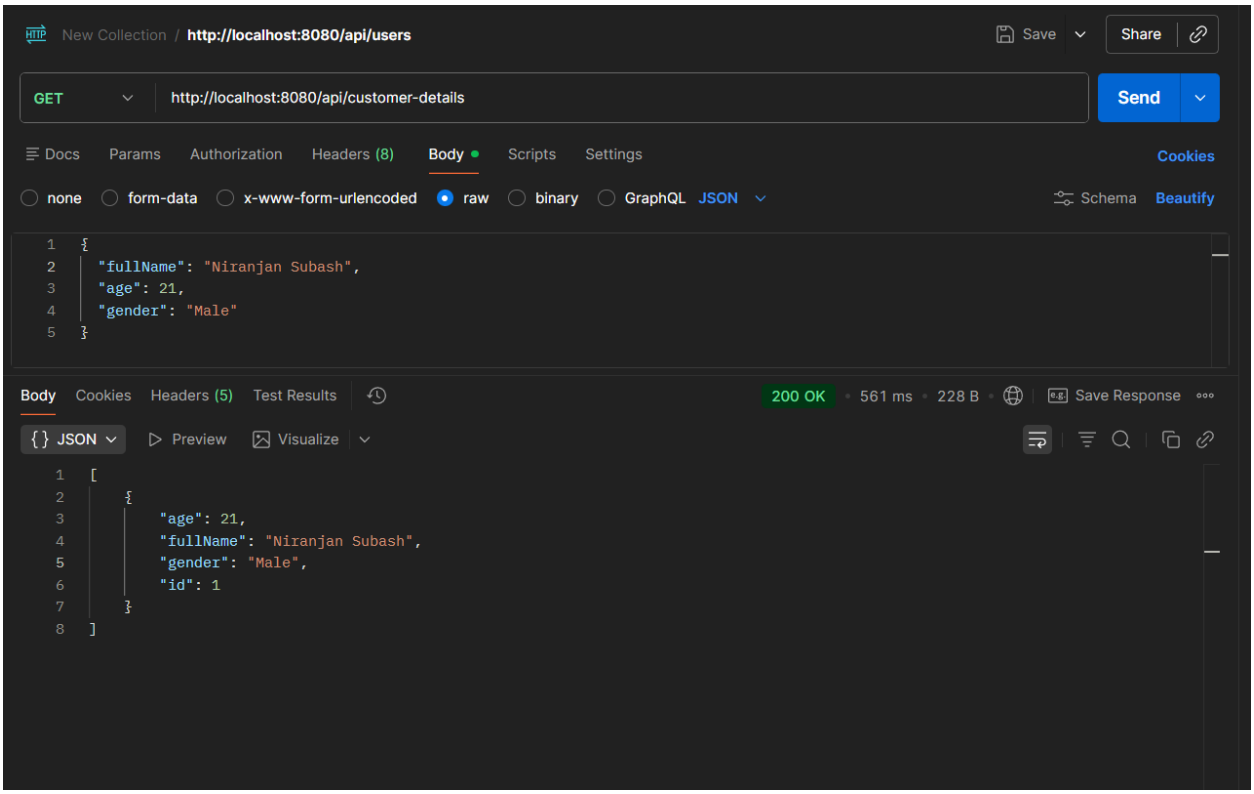


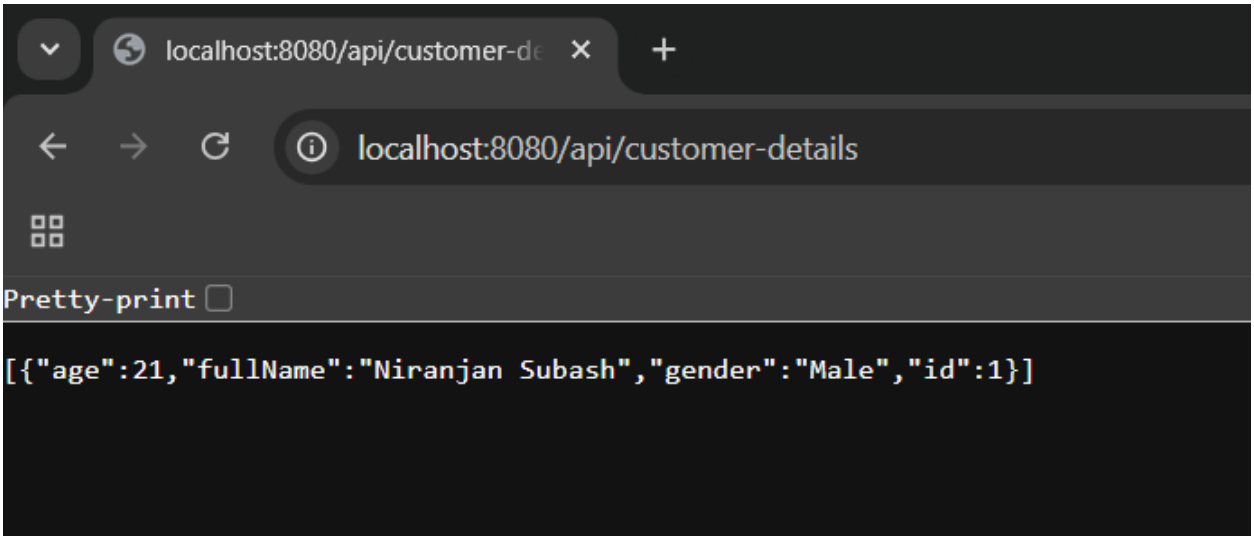
```
mysql> select * from user;
+----+-----------------+------+
| id | email           | name |
+----+-----------------+------+
|  1 | PKA@manipal.edu | PKA  |
+----+-----------------+------+
1 row in set (0.00 sec)
```

Lab Exercises

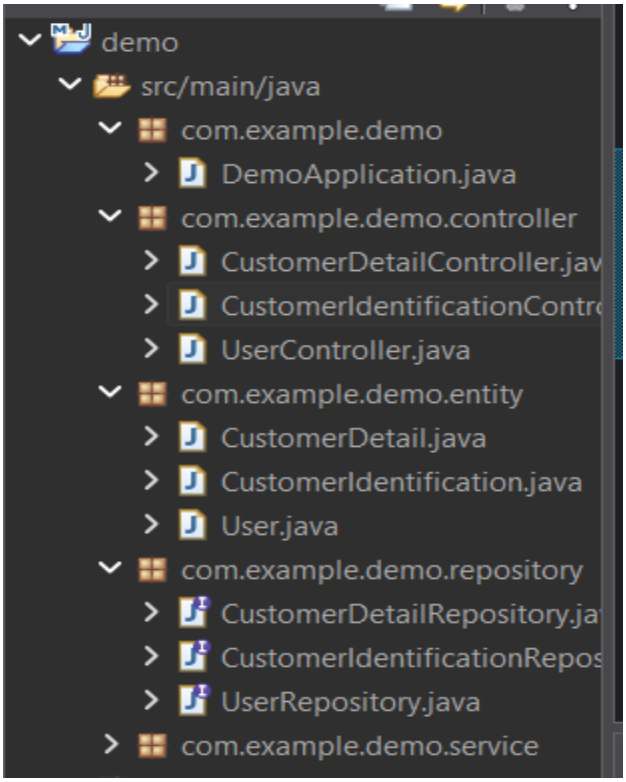
1. Establish Spring Boot MySQL Connection with the CustomerDetail table.

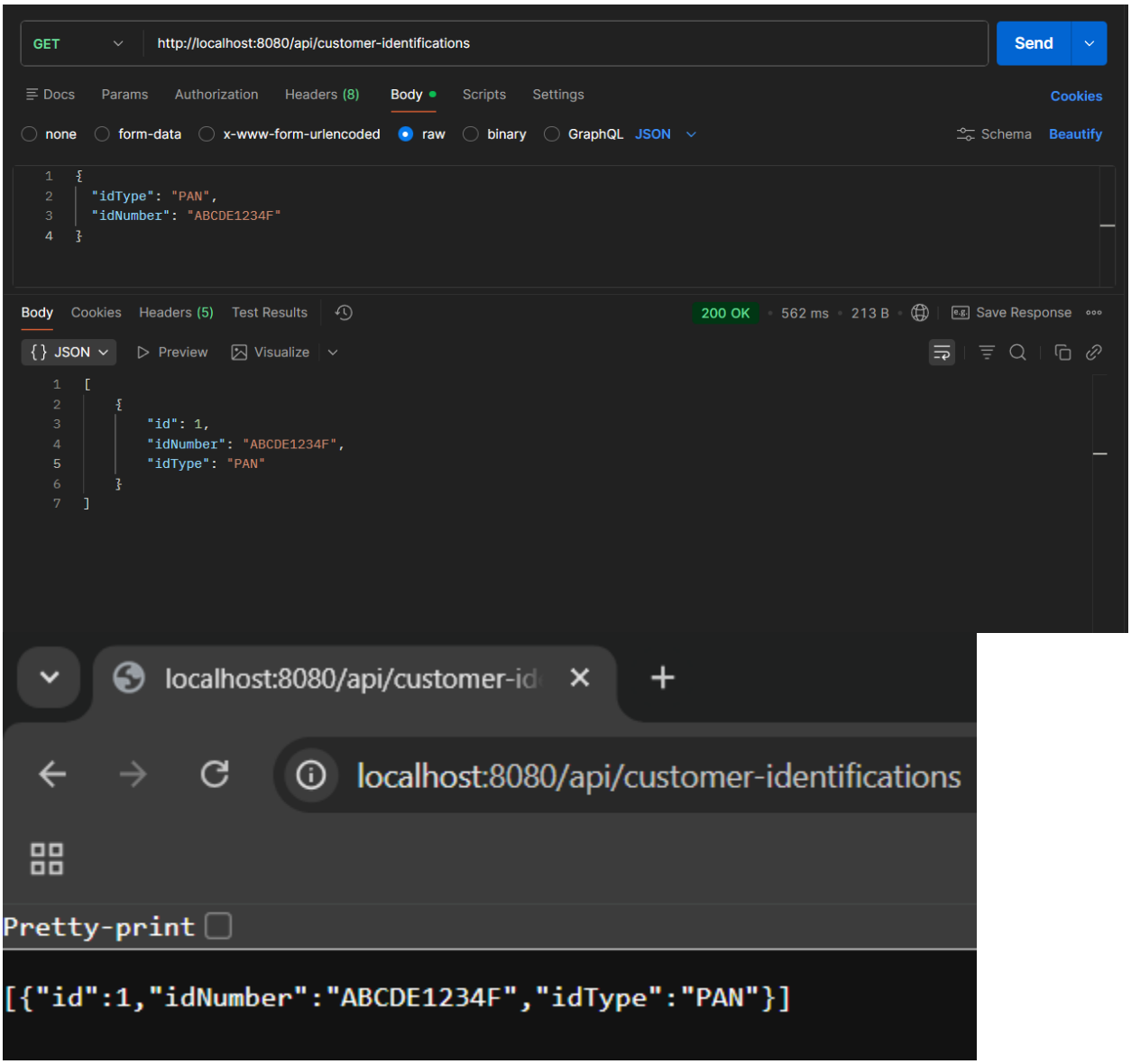




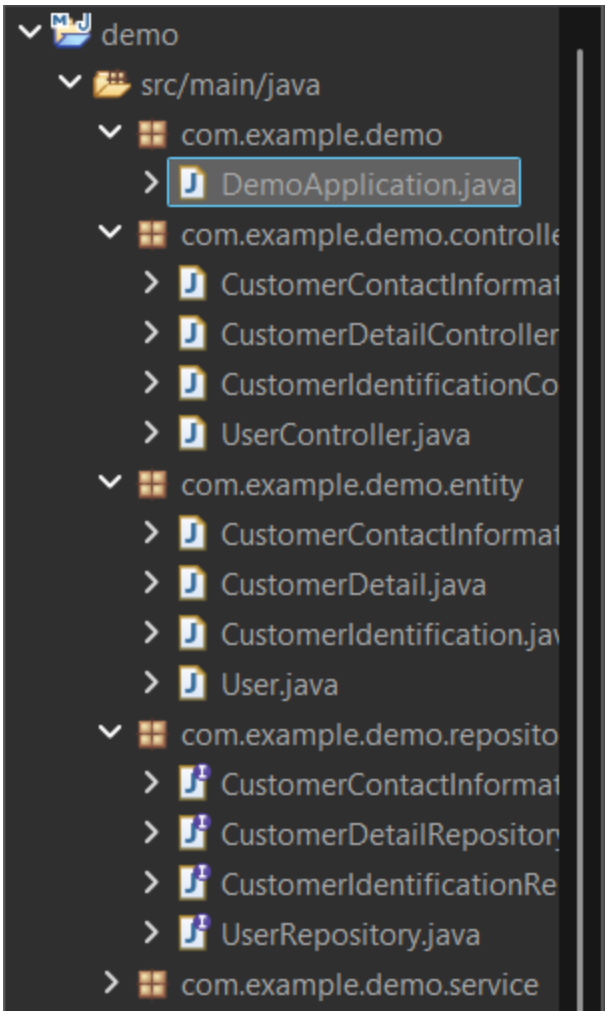


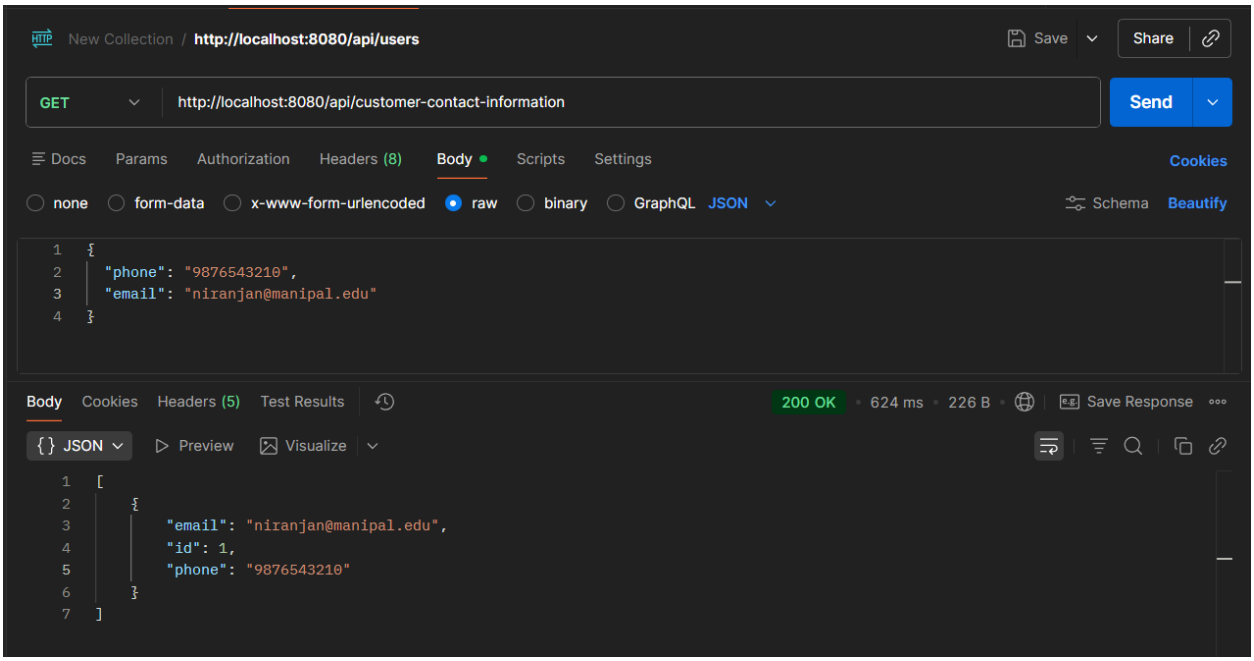
2. Establish Spring Boot MySQL Connection with CustomerIdentification table

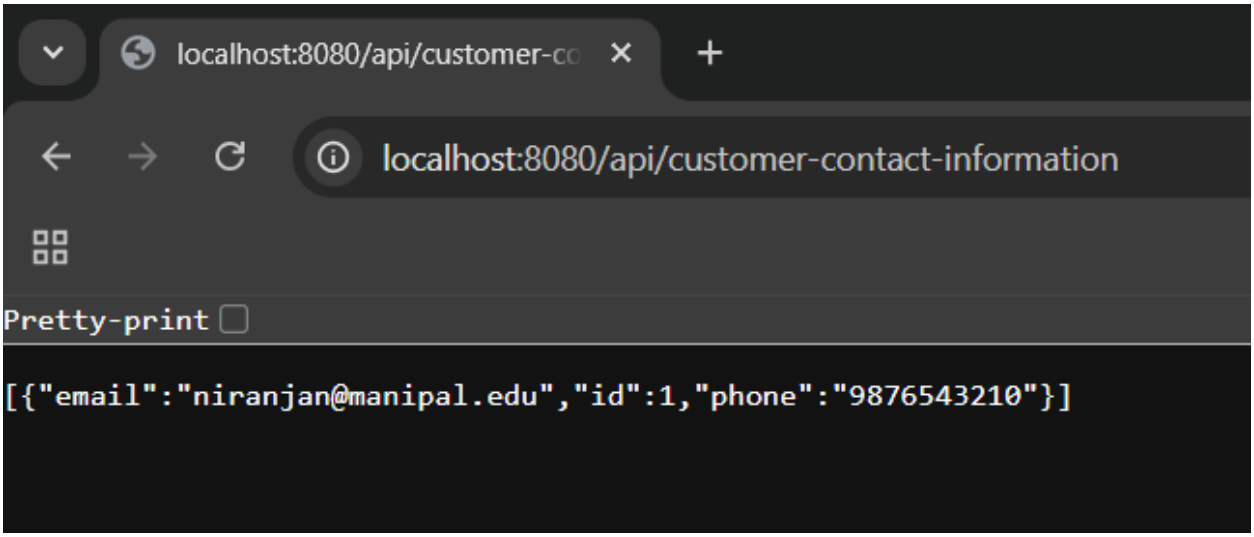




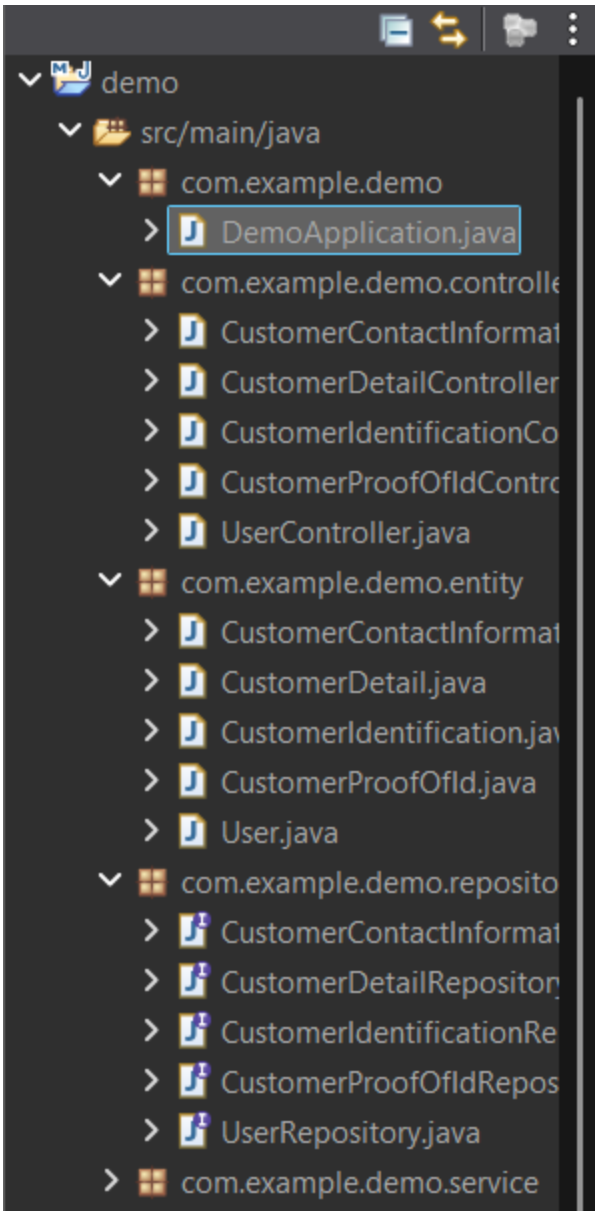
3. Establish Spring Boot MySQL Connection with CustomerContactInformation table

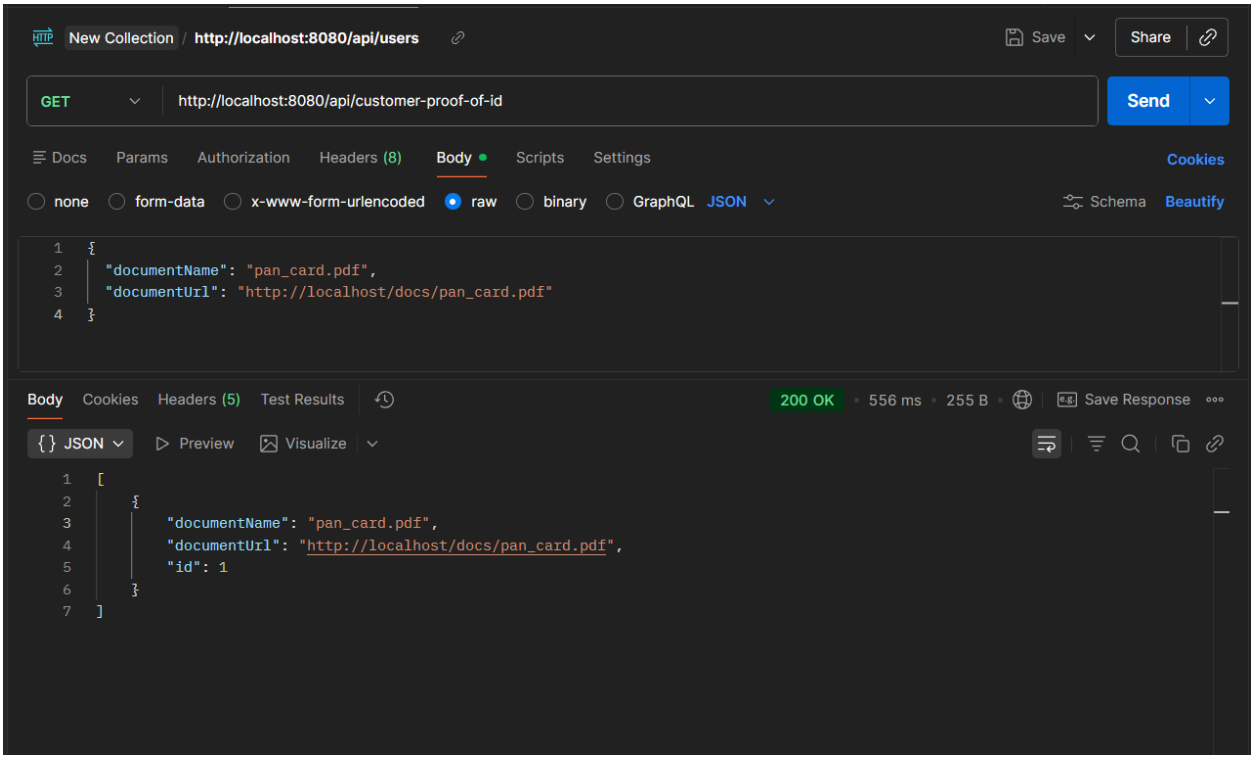

demo
src/main/java
com.example.demo
DemoApplication.java
com.example.demo.controlle
CustomerContactInformat
CustomerDetailController
CustomerIdentificationCo
UserController.java
com.example.demo.entity
CustomerContactInformat
CustomerDetail.java
CustomerIdentification.jav
User.java
com.example.demo.reposito
CustomerContactInformat
CustomerDetailRepositor
CustomerIdentificationRe
UserRepository.java
com.example.demo.service

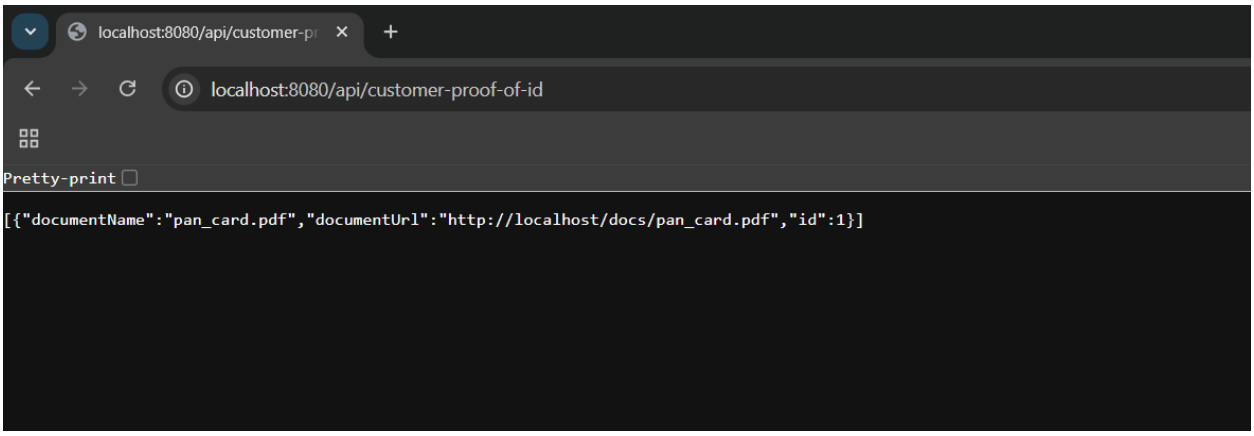

New Collection / http://localhost:8080/api/users
Save
Share
GET
http://localhost:8080/api/customer-contact-information
Send
Docs
Params
Authorization
Headers (8)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"phone": "9876543210",
"email": "niranjan@manipal.edu"
}
Body
Cookies
Headers (5)
Test Results
200 OK
624 ms
226 B
Save Response
JSON
Preview
Visualize
[
{
"email": "niranjan@manipal.edu",
"id": 1,
"phone": "9876543210"
}
]



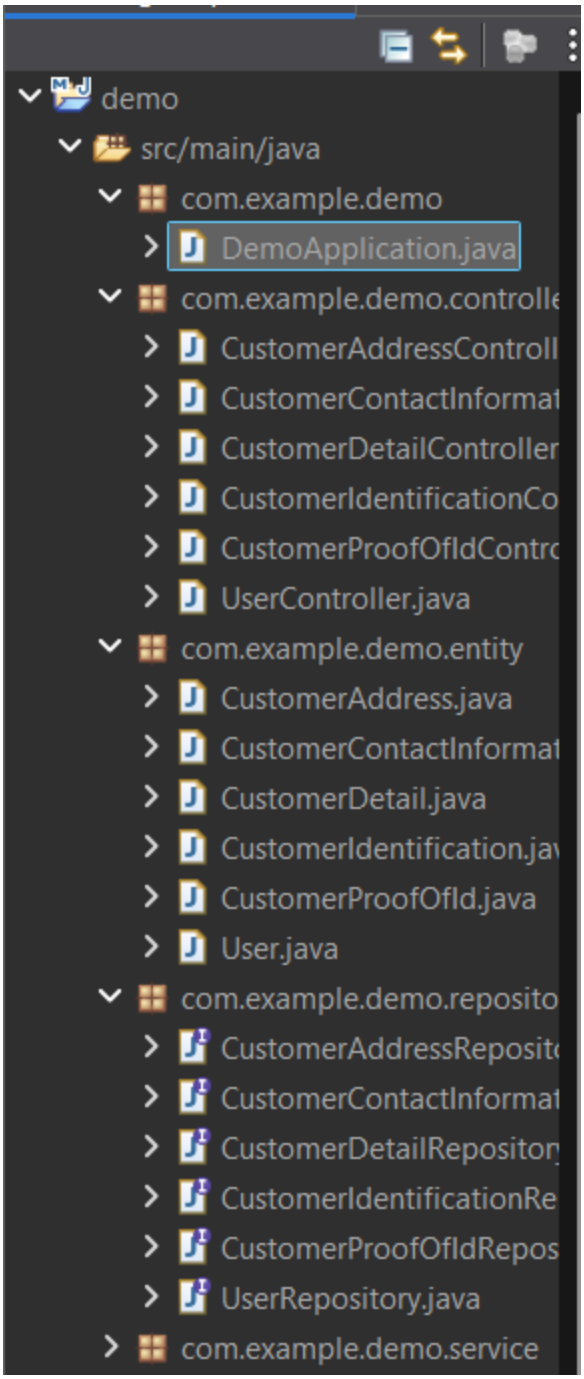
4. Establish Spring Boot MySQL Connection with CustomerProofOfId table


demo
src/main/java
com.example.demo
DemoApplication.java
com.example.demo.controlle
CustomerContactInformat
CustomerDetailController
CustomerIdentificationCo
CustomerProofOfIdContrc
UserController.java
com.example.demo.entity
CustomerContactInformat
CustomerDetail.java
CustomerIdentification.jav
CustomerProofOfId.java
User.java
com.example.demo.reposito
CustomerContactInformat
CustomerDetailRepositor
CustomerIdentificationRe
CustomerProofOfIdRepos
UserRepository.java
com.example.demo.service





5. Establish Spring Boot MySQL Connection with CustomerAddress


demo
src/main/java
com.example.demo
DemoApplication.java
com.example.demo.controlle
CustomerAddressControll
CustomerContactInformat
CustomerDetailController
CustomerIdentificationCo
CustomerProofOfIdContrc
UserController.java
com.example.demo.entity
CustomerAddress.java
CustomerContactInformat
CustomerDetail.java
CustomerIdentification.jav
CustomerProofOfId.java
User.java
com.example.demo.reposito
CustomerAddressReposit
CustomerContactInformat
CustomerDetailRepositor
CustomerIdentificationRe
CustomerProofOfIdRepos
UserRepository.java
com.example.demo.service

New Collection / http://localhost:8080/api/users

Save | Share

GET http://localhost:8080/api/customer-addresses Send

Docs | Params | Authorization | Headers (8) | Body | Scripts | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

```
2  "houseNo": "12A",
3  "street": "MG Road",
4  "city": "Manipal",
5  "state": "Karnataka",
6  "pincode": "576104"
7  }
```

Body | Cookies | Headers (5) | Test Results

200 OK • 637 ms • 265 B • Save Response

{} JSON | Preview | Visualize

```
1  [
2      {
3          "city": "Manipal",
4          "houseNo": "12A",
5          "id": 1,
6          "pincode": "576104",
7          "state": "Karnataka",
8          "street": "MG Road"
9      }
10 ]
```

localhost:8080/api/customer-a

localhost:8080/api/customer-addresses

Pretty-print

[{"city":"Manipal","houseNo":"12A","id":1,"pincode":"576104","state":"Karnataka","street":"MG Road"}]

LAB 5 & 6:

Experiment: Spring Boot with MySQL – REST APIs using PUT, POST, GET

Aim

To develop Spring Boot applications using RESTful web services to perform **POST, GET, and PUT** operations on different customer-related tables using **MySQL** as the backend database.

Lab Exercises

1. Write a spring boot application for PUT, POST, and GET customer name

APIs Implemented

POST
POST: http://localhost:8080/api/customer-detail

New Collection / http://localhost:8080/api/users

Save | Share

POST http://localhost:8080/api/customer-details Send

Docs | Params | Authorization | Headers (8) | Body | Scripts | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

```
1 {
2   "fullName": "Martin Scorsese",
3   "age": 43,
4   "gender": "Male"
5 }
```

Body | Cookies | Headers (5) | Test Results | 200 OK • 45 ms • 226 B | Save Response

JSON | Preview | Visualize

```
1 {
2     "age": 43,
3     "fullName": "Martin Scorsese",
4     "gender": "Male",
5     "id": 4
6 }
```

GET

GET: http://localhost:8080/api/customer-detail

New Collection / http://localhost:8080/api/users

Save | Share

GET http://localhost:8080/api/customer-details Send

Docs | Params | Authorization | Headers (8) | Body | Scripts | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

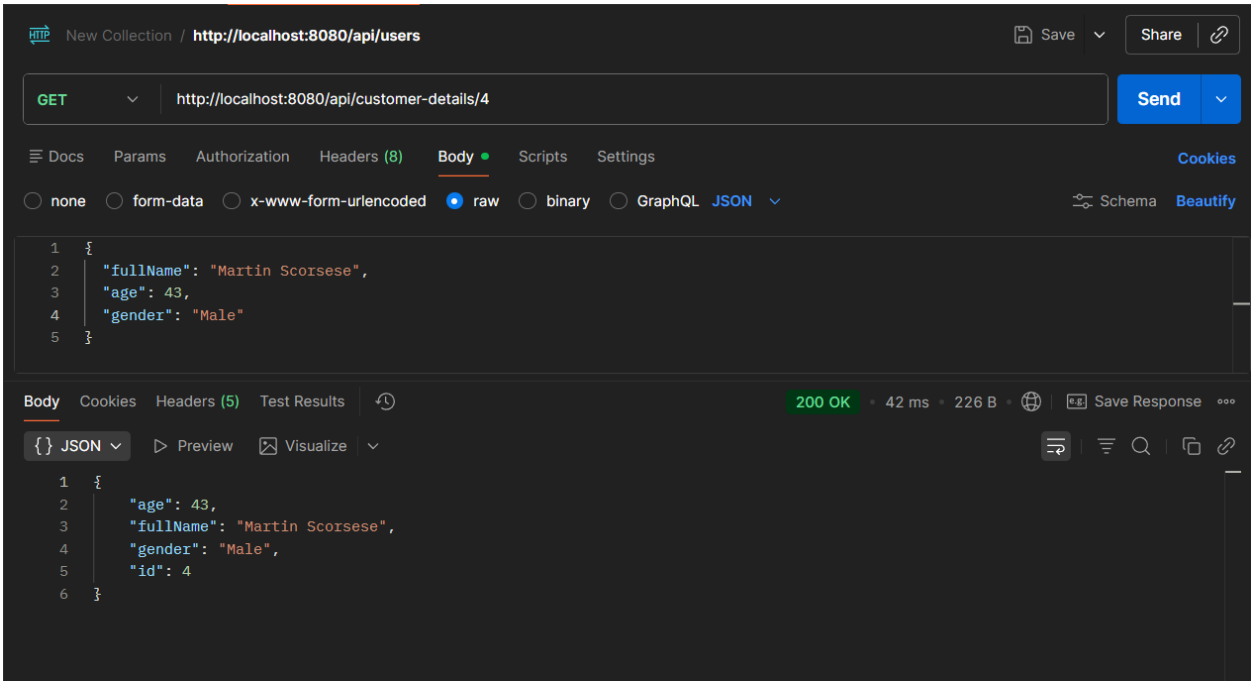
```
1 {
2   "fullName": "Martin Scorsese",
3   "age": 43,
4   "gender": "Male"
5 }
```

Body | Cookies | Headers (5) | Test Results | 200 OK • 78 ms • 291 B | Save Response

JSON | Preview | Visualize

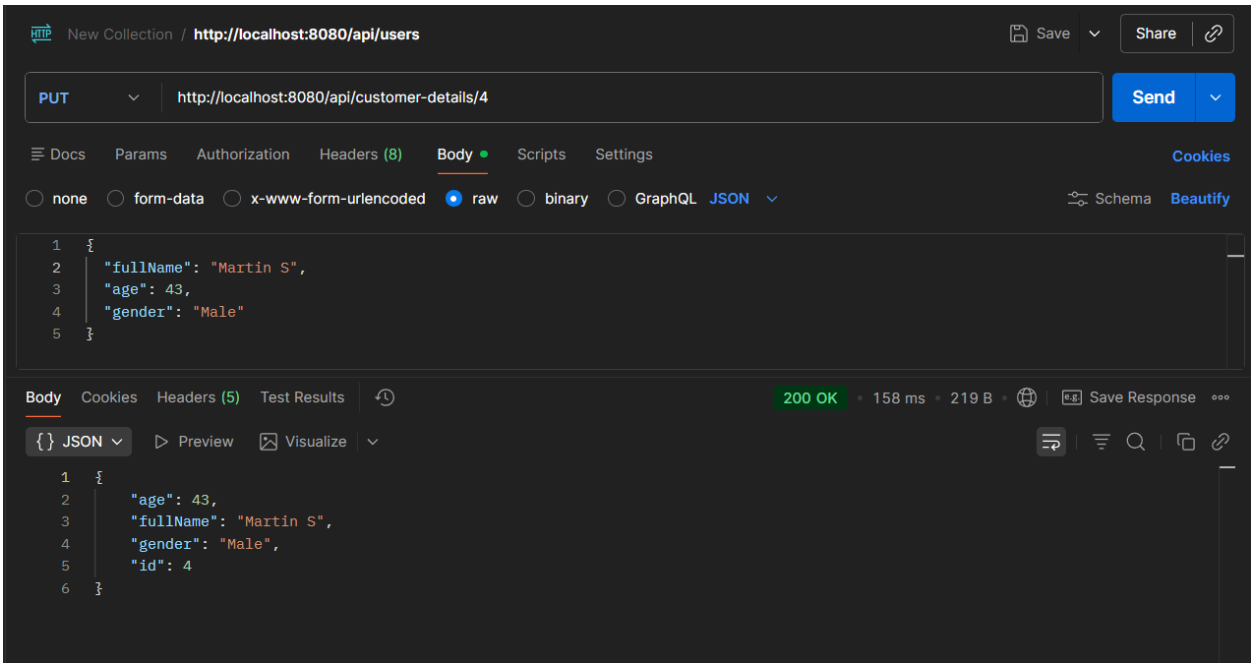
```
1 [
2     {
3         "age": 21,
4         "fullName": "Niranjan Subash",
5         "gender": "Male",
6         "id": 1
7     },
8     {
9         "age": 43,
10        "fullName": "Martin Scorsese",
11        "gender": "Male",
12        "id": 4
13    }
14 ]
```

GET: http://localhost:8080/api/customer-detail/{id}



PUT

PUT: http://localhost:8080/api/customer-detail/{id}



2. Write a spring boot application for PUT, POST, and GET customer details

APIs Implemented

POST
POST http://localhost:8080/api/customer-detail

GET
GET http://localhost:8080/api/customer-detail
GET http://localhost:8080/api/customer-detail/{id}

PUT
PUT http://localhost:8080/api/customer-detail/{id}

(SAME AS THE PREVIOUS QUESTION)

3. Write a spring boot application for PUT, POST and GET customer identification

APIs Implemented

POST
POST http://localhost:8080/api/customer-identification

New Collection / http://localhost:8080/api/users

Save | Share

POST http://localhost:8080/api/customer-identifications Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "idType": "PAN",
3   "idNumber": "ABCDEF123456"
4 }
```

Body Cookies Headers (5) Test Results 200 OK 29 ms 213 B Save Response

{} JSON Preview Visualize

```
1 {
2     "id": 4,
3     "idNumber": "ABCDEF123456",
4     "idType": "PAN"
5 }
```

GET

GET http://localhost:8080/api/customer-identification

New Collection / http://localhost:8080/api/users

Save | Share

GET http://localhost:8080/api/customer-identifications Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

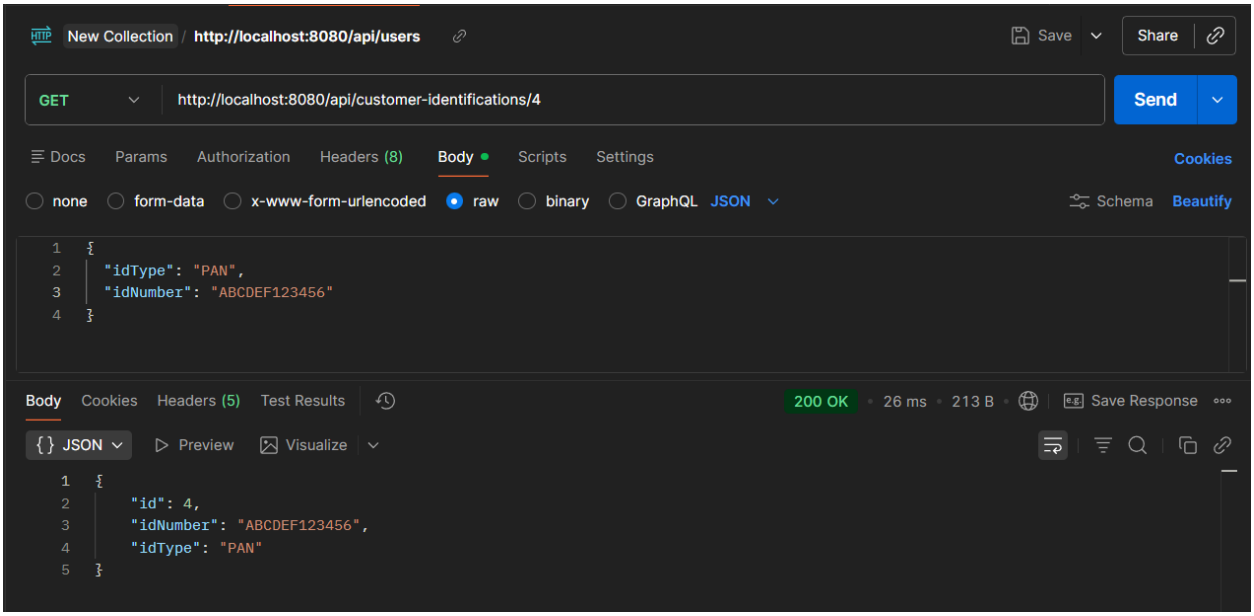
```
1 {
2   "idType": "PAN",
3   "idNumber": "ABCDEF123456"
4 }
```

Body Cookies Headers (5) Test Results 200 OK 29 ms 263 B Save Response

{} JSON Preview Visualize

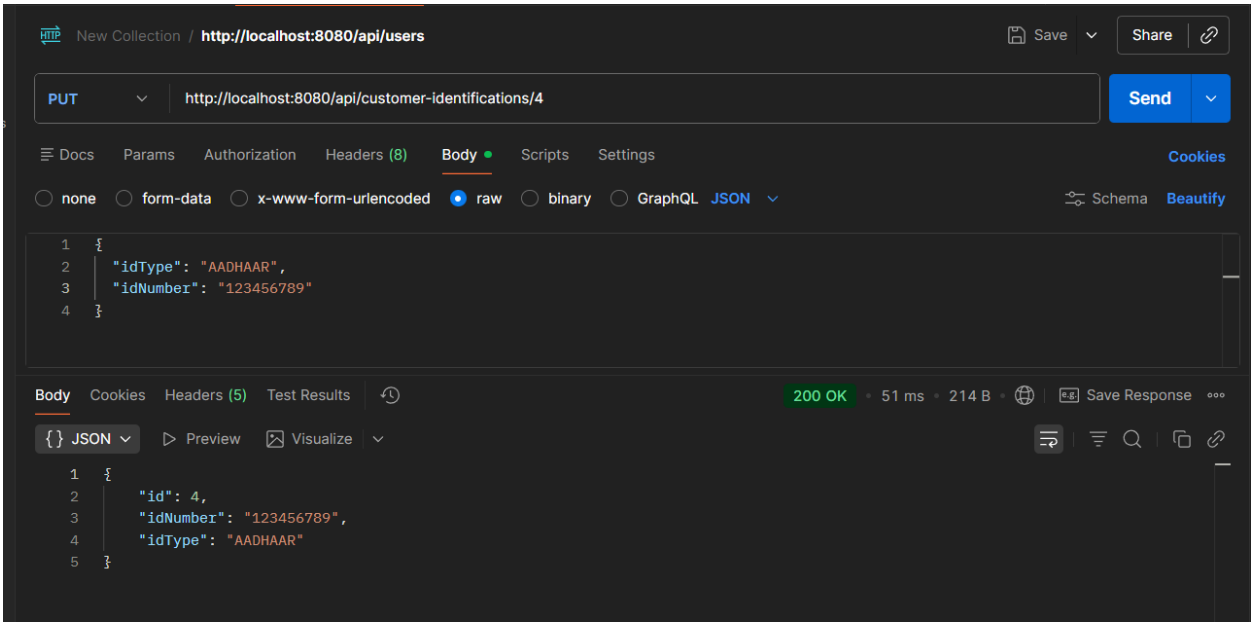
```
1  [
2      {
3          "id": 1,
4          "idNumber": "ABCDE1234F",
5          "idType": "PAN"
6      },
7      {
8          "id": 4,
9          "idNumber": "ABCDEF123456",
10         "idType": "PAN"
11     }
12 ]
```

GET http://localhost:8080/api/customer-identification/{id}



PUT

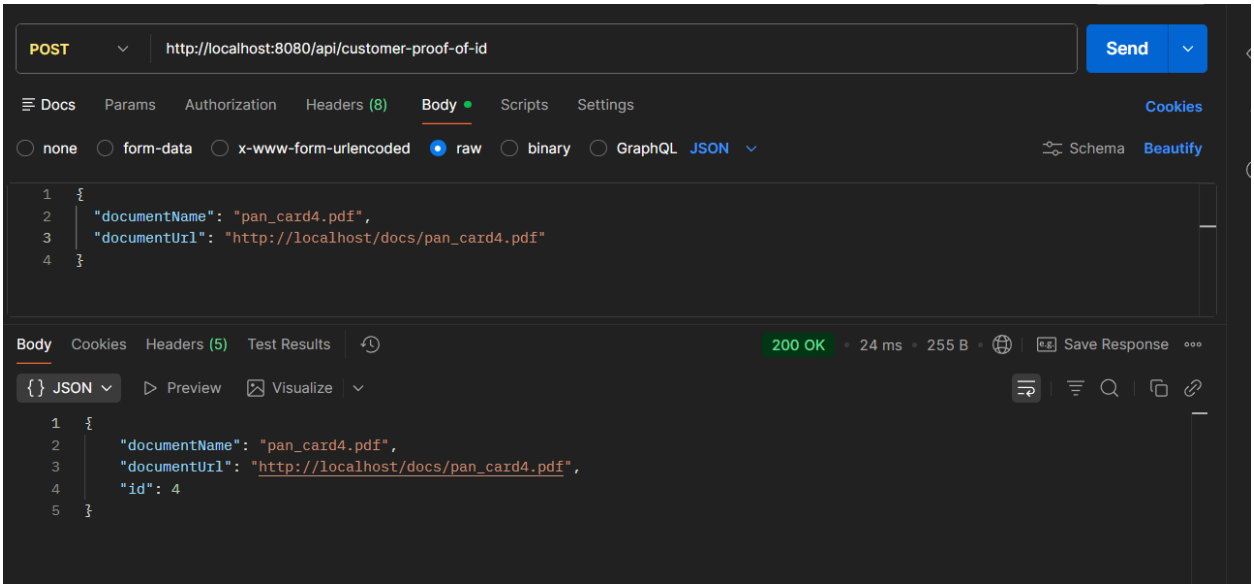
PUT http://localhost:8080/api/customer-identification/{id}



4. Write a spring boot application for PUT, POST, and GET customer proof of id

APIs Implemented

POST
POST http://localhost:8080/api/customer-proof-of-id



GET
GET http://localhost:8080/api/customer-proof-of-id

New Collection / http://localhost:8080/api/users

Save | Share

GET http://localhost:8080/api/customer-proof-of-id Send

Docs | Params | Authorization | Headers (8) | Body | Scripts | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

```
1 {
2   "documentName": "pan_card4.pdf",
3   "documentUrl": "http://localhost/docs/pan_card4.pdf"
4 }
```

Body | Cookies | Headers (5) | Test Results | 200 OK • 30 ms • 347 B | Save Response

JSON | Preview | Visualize

```
1  [
2      {
3          "documentName": "pan_card.pdf",
4          "documentUrl": "http://localhost/docs/pan_card.pdf",
5          "id": 1
6      },
7      {
8          "documentName": "pan_card4.pdf",
9          "documentUrl": "http://localhost/docs/pan_card4.pdf",
10         "id": 4
11     }
12 ]
```

GET http://localhost:8080/api/customer-proof-of-id/{id}

New Collection / http://localhost:8080/api/users

Save | Share

GET http://localhost:8080/api/customer-proof-of-id/4 Send

Docs | Params | Authorization | Headers (8) | Body | Scripts | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

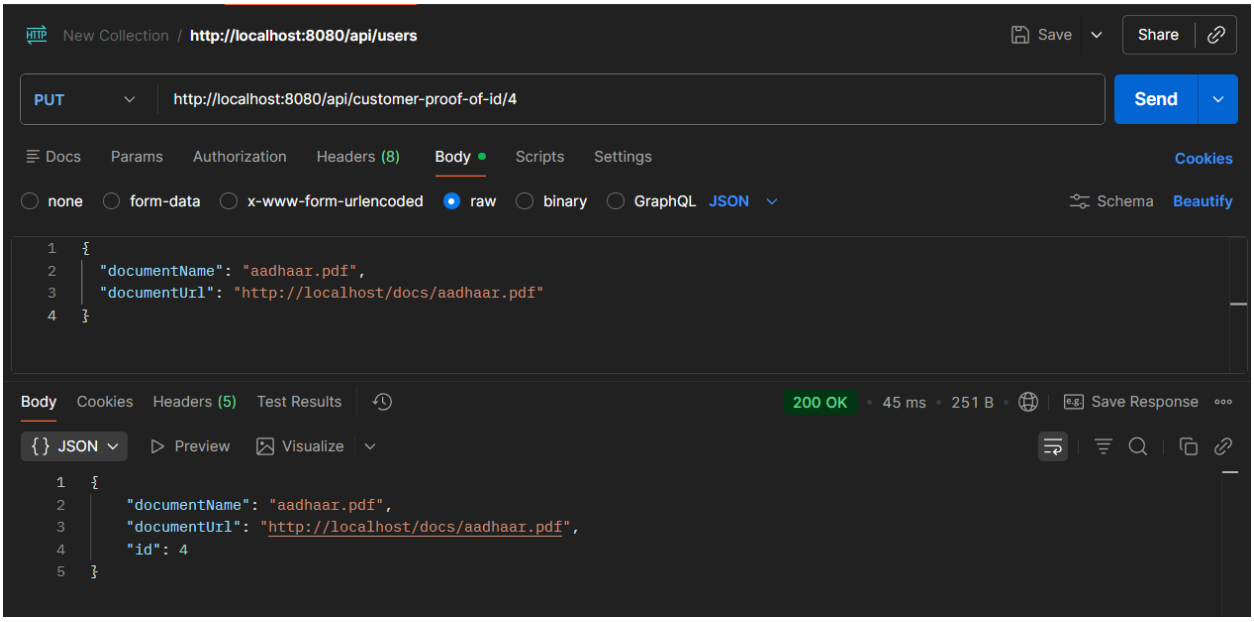
```
1 {
2   "documentName": "pan_card4.pdf",
3   "documentUrl": "http://localhost/docs/pan_card4.pdf"
4 }
```

Body | Cookies | Headers (5) | Test Results | 200 OK • 21 ms • 255 B | Save Response

JSON | Preview | Visualize

```
1 {
2     "documentName": "pan_card4.pdf",
3     "documentUrl": "http://localhost/docs/pan_card4.pdf",
4     "id": 4
5 }
```

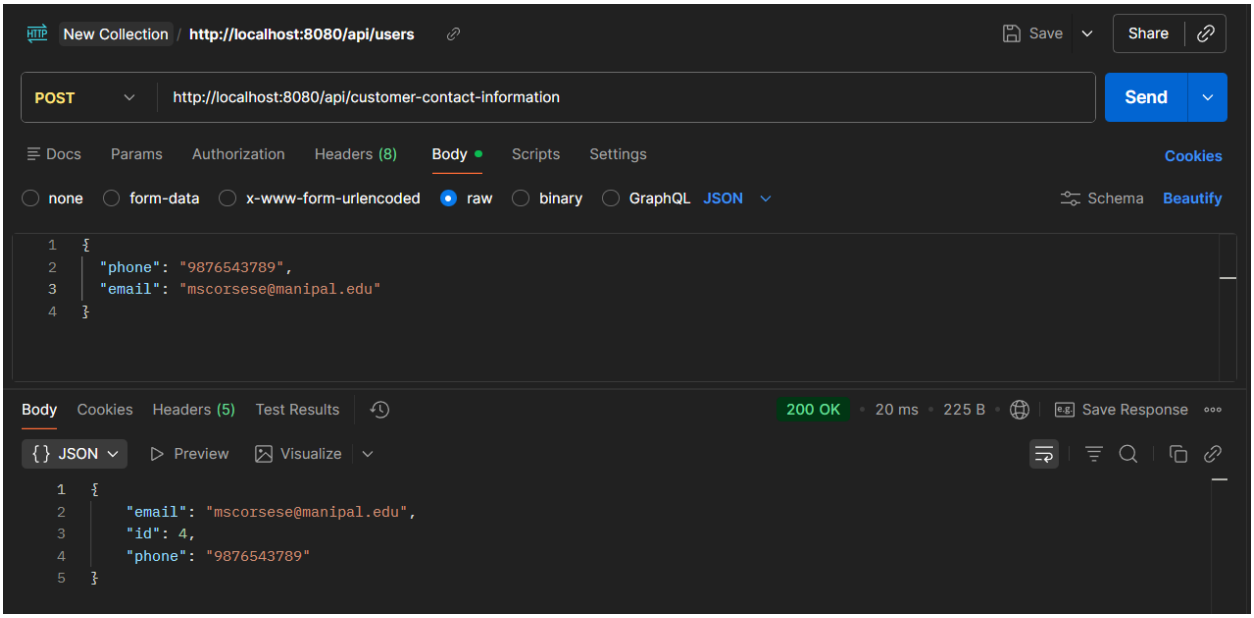
PUT
PUT http://localhost:8080/api/customer-proof-of-id/{id}



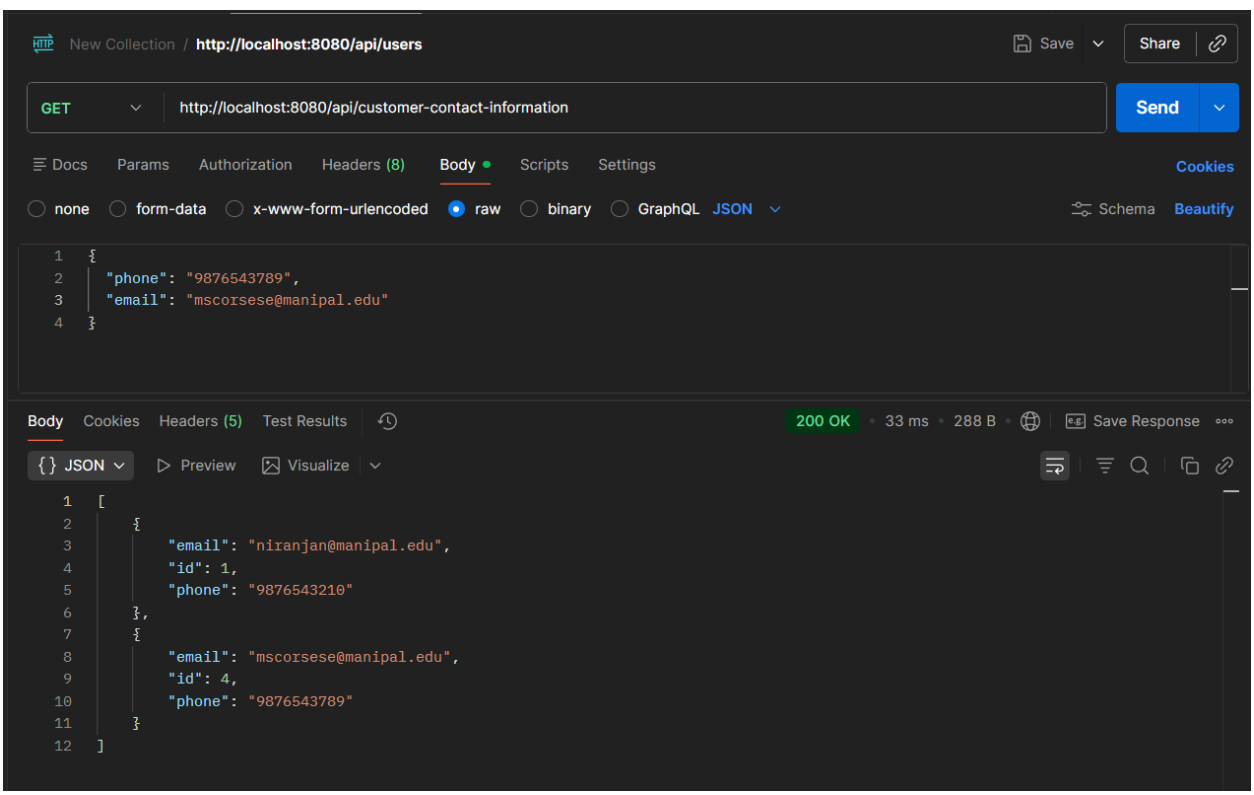
5. Write a spring boot application for PUT, POST, and GET customer contact details

APIs Implemented

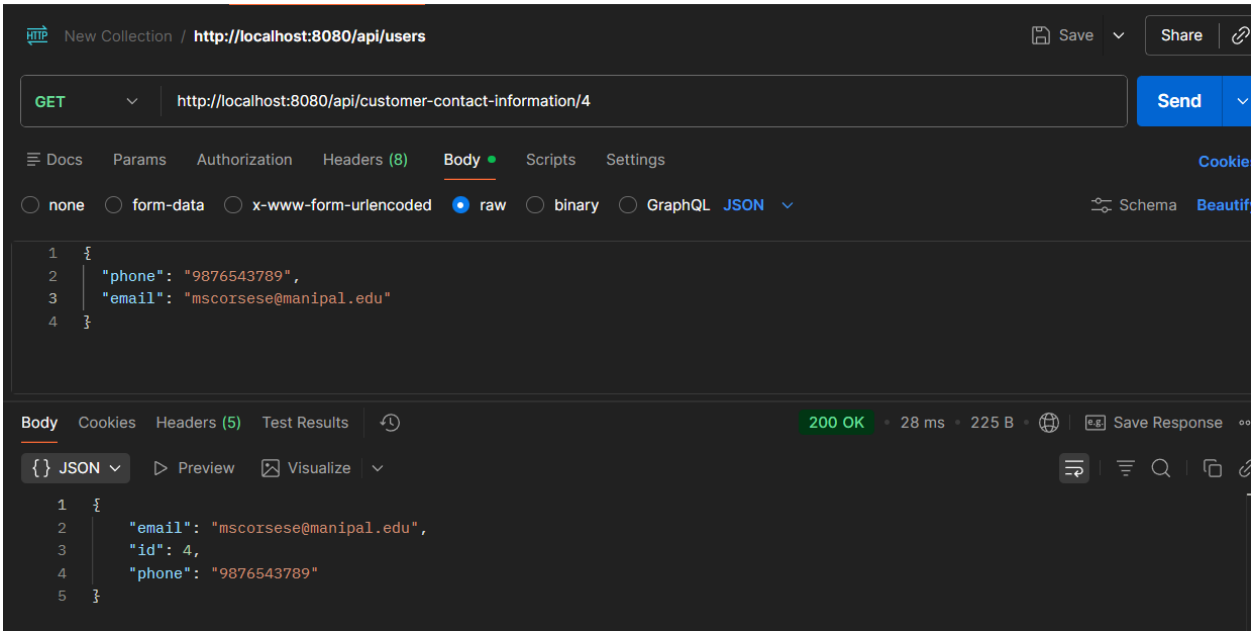
POST
POST http://localhost:8080/api/customer-contact-information



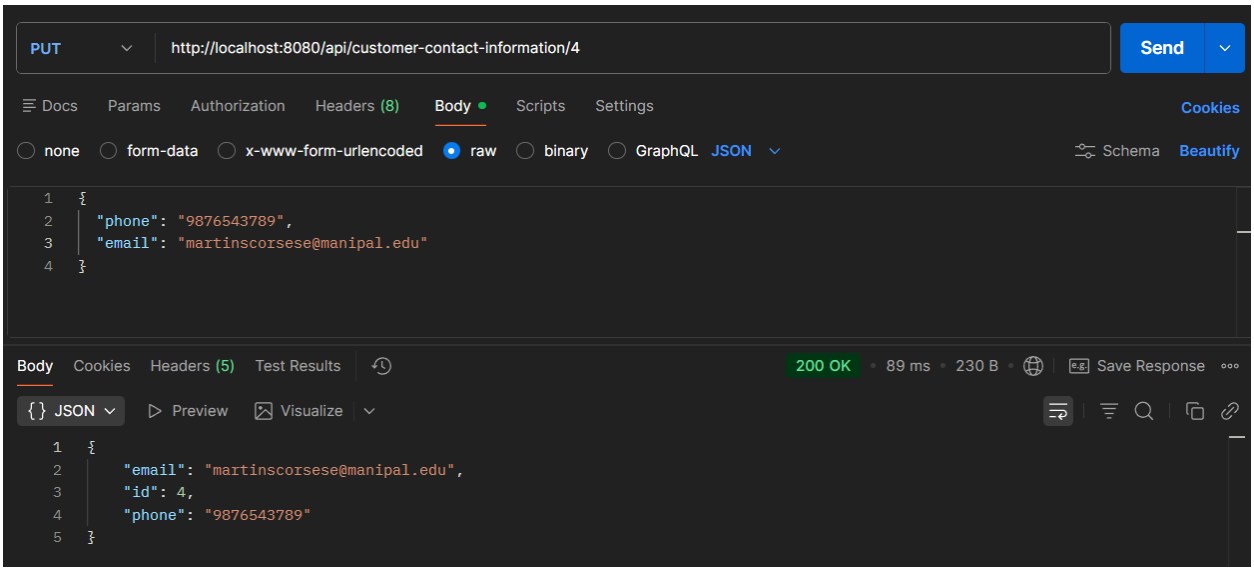
GET
GET http://localhost:8080/api/customer-contact-information



GET http://localhost:8080/api/customer-contact-information/{id}



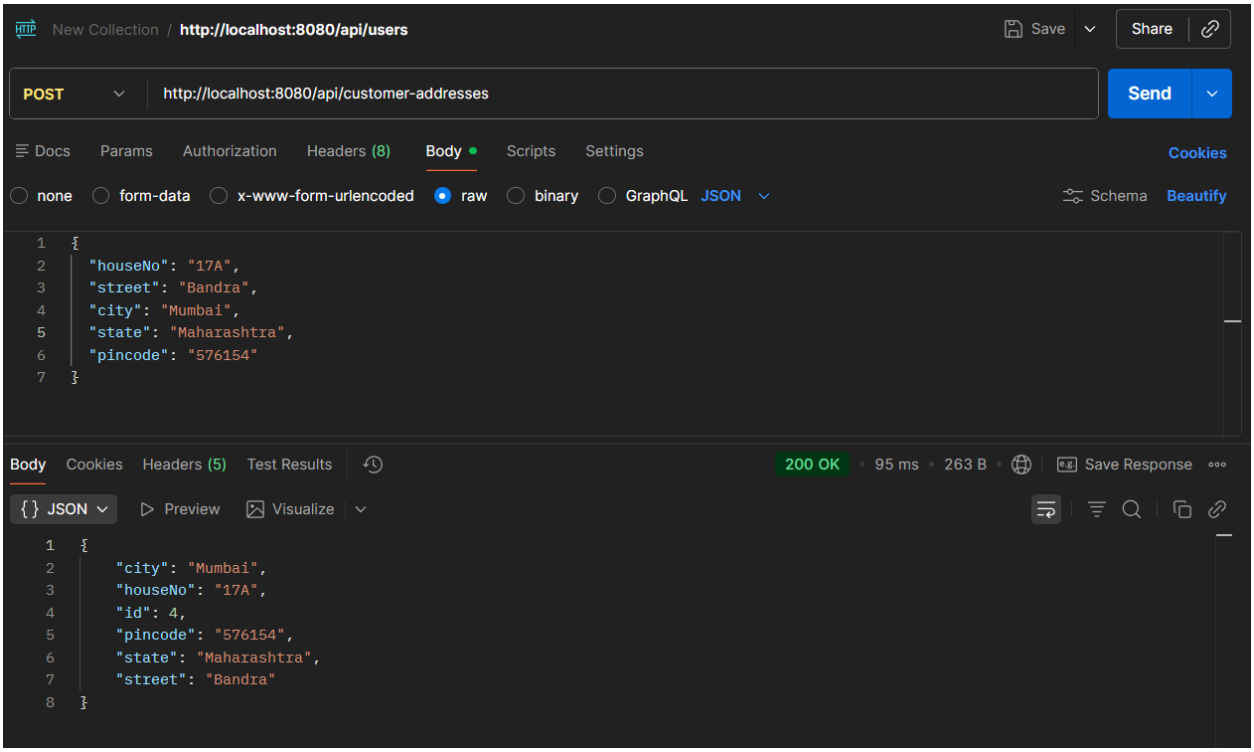
PUT
PUT http://localhost:8080/api/customer-contact-information/{id}



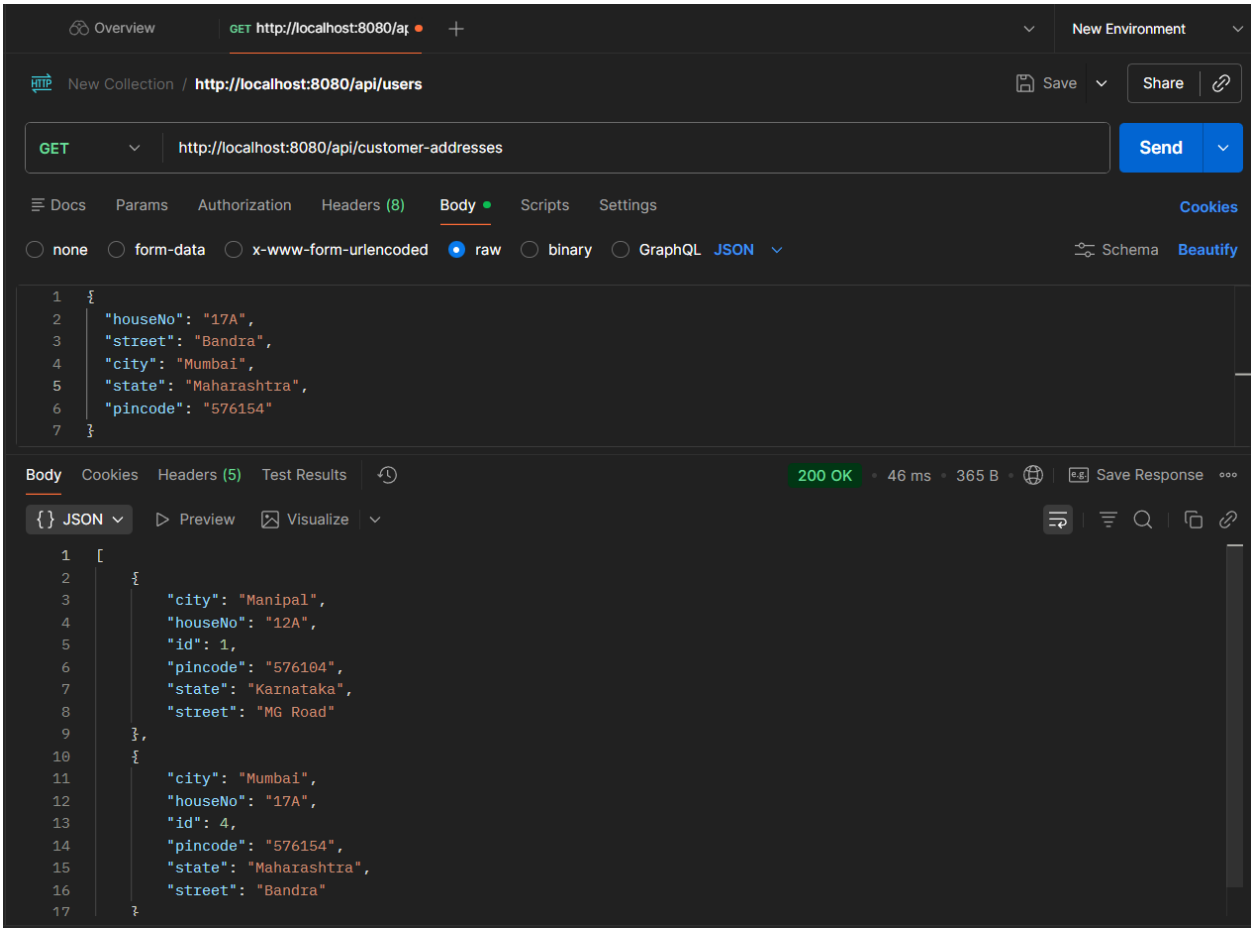
6. Write a spring boot application for PUT, POST, and GET customer address

APIs Implemented

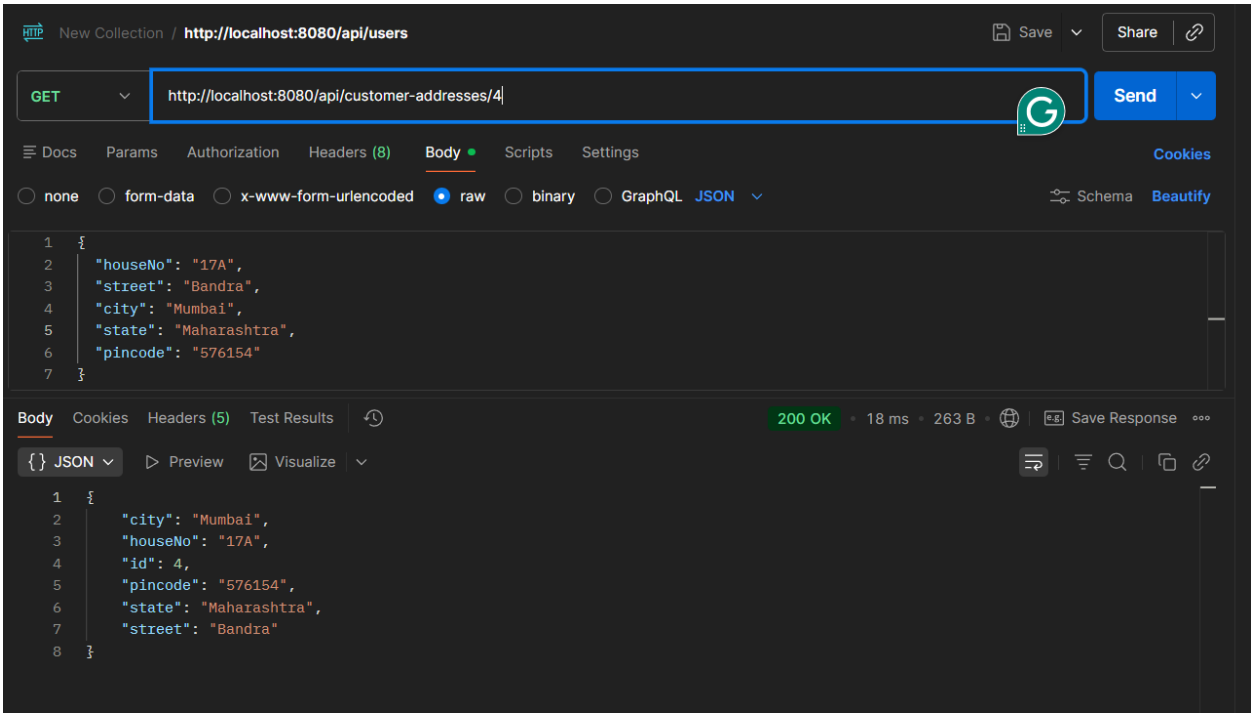
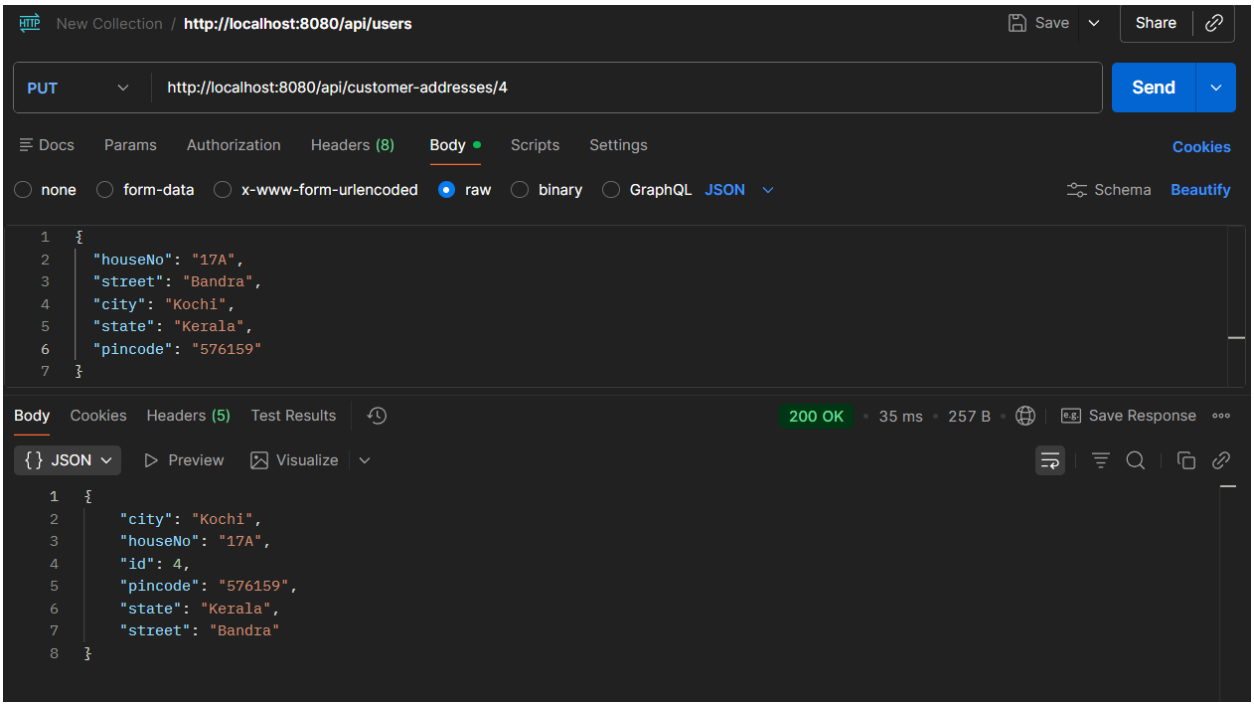
POST
POST http://localhost:8080/api/customer-address



GET
GET http://localhost:8080/api/customer-address



GET http://localhost:8080/api/customer-address/{id}



PUT
PUT http://localhost:8080/api/customer-address/{id}